

数据库期中复习

重点看2、3、7章习题

第一章

Database System(DBS) = DB + DBM + Users/Adminiers

实现数据管理功能

信息/数据管理（management）■数据的分类、收集、组织、编码、存储、检索和维护

Database Application System(DBAS) = DB + DBM + Application programs + Users/Adminiers

实现数据处理功能

信息/数据处理（processing）■对数据进行分类、收集、组织、存储，进而从已有数据出发，抽取或推导出新的数据/信息

信息/数据处理与信息/数据管理的区别

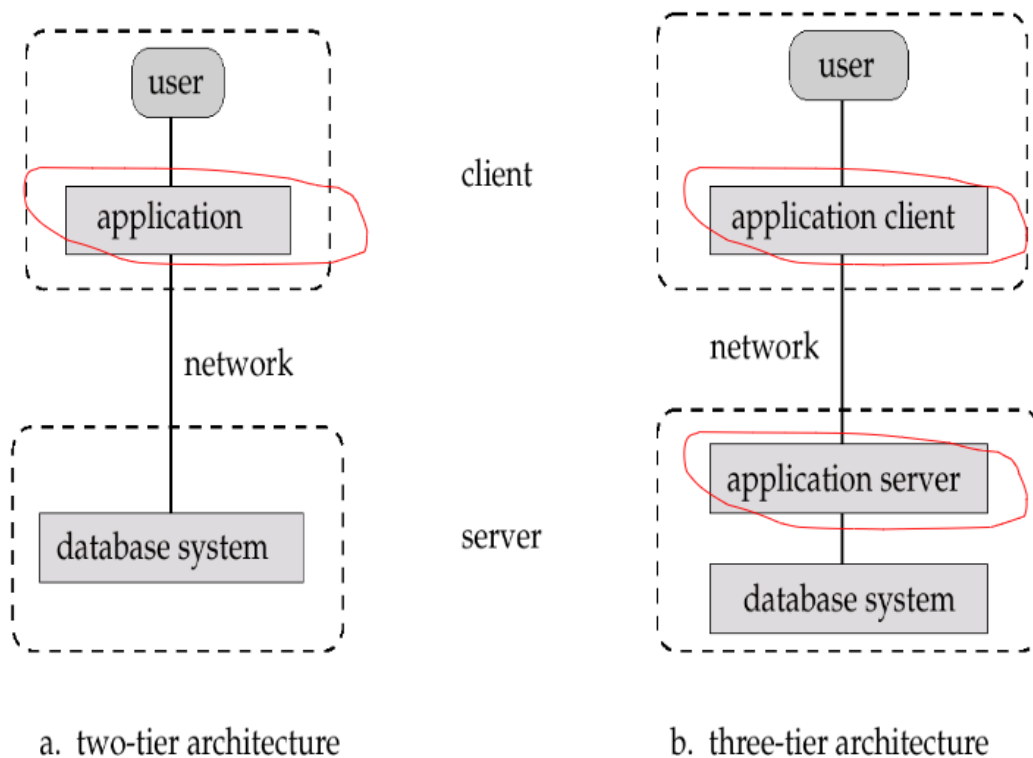
■数据管理侧重于对数据如何进行组织和存储，并根据用户需要提供对数据的访问支持

■数据处理除了具有数据管理功能外，还可以对通过数据管理得到的数据进行进一步深加工，从中获取新的数据、信息

■In DBAS, the application programs are responsible for information or data processing, e.g. data mining

■Two-tier architecture■e.g. client programs using ODBC/JDBC to communicate with a database•Three-tier architecture

■e.g. web-based applications, and applications built using “middleware”



integrity constraints = data semantics + consistency constraints(完整性约束=一致性约束+数据语义)

data manipulation language(DML)数据操作语言

数据操作语言包括retrieve、insert、delete、modify（查询、插入、删除、更改）

数据操作语言分为procedural过程性、nonprocedural(declarative)说明性的。

过程性用于用户指明需要什么数据、怎么获取这些数据

说明性只用于用户指明需要什么数据即可

数据操作语言又分为两类：**pure**（用来证明的，关系代数、元组关系演算、域关系演算）和**commercial**（商业的，用于商业系统，SQL，SQL是说明性语言）

应用程序通常通过“语言扩展”之一访问数据库：①以允许嵌入式SQL应用程序接口（例如ODBC / JDBC）；②该接口允许将SQL查询发送到数据库。

查询

```
select name
from instructor
where dept_name= 'Comp. Sci.' and salary > 80000
```

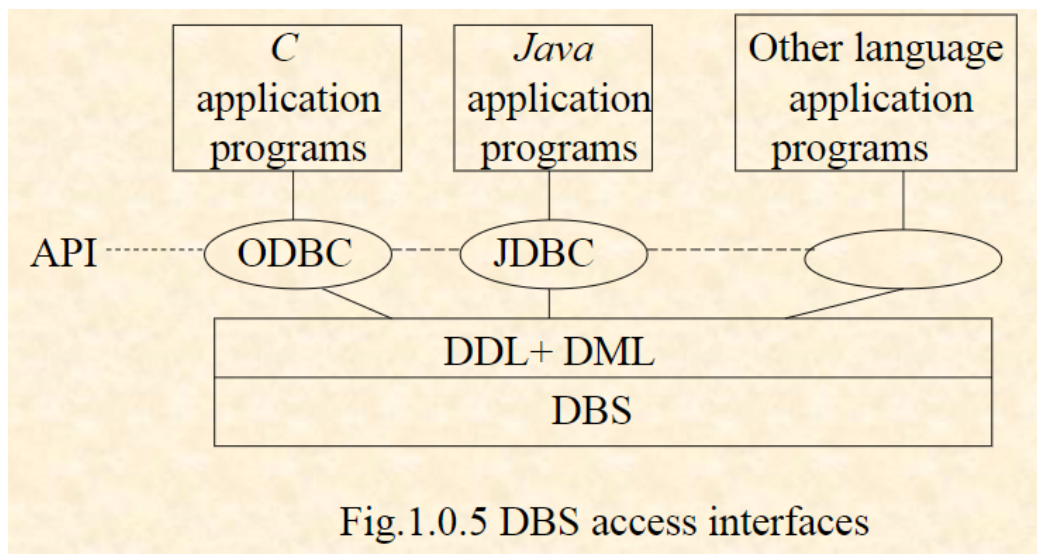
建表

```
create table instructor(
ID char(5),
name varchar(20),
dept_name varchar(20),
salary numeric(8,2))
```

data definition language(DDL)数据定义语言

DDL编译器生成一组存储在数据字典中的表模板。数据字典包含元数据(即关于数据的数据): 数据库模式, 完整性约束——主键(ID唯一标识指使者), 授权(谁可以访问什么)

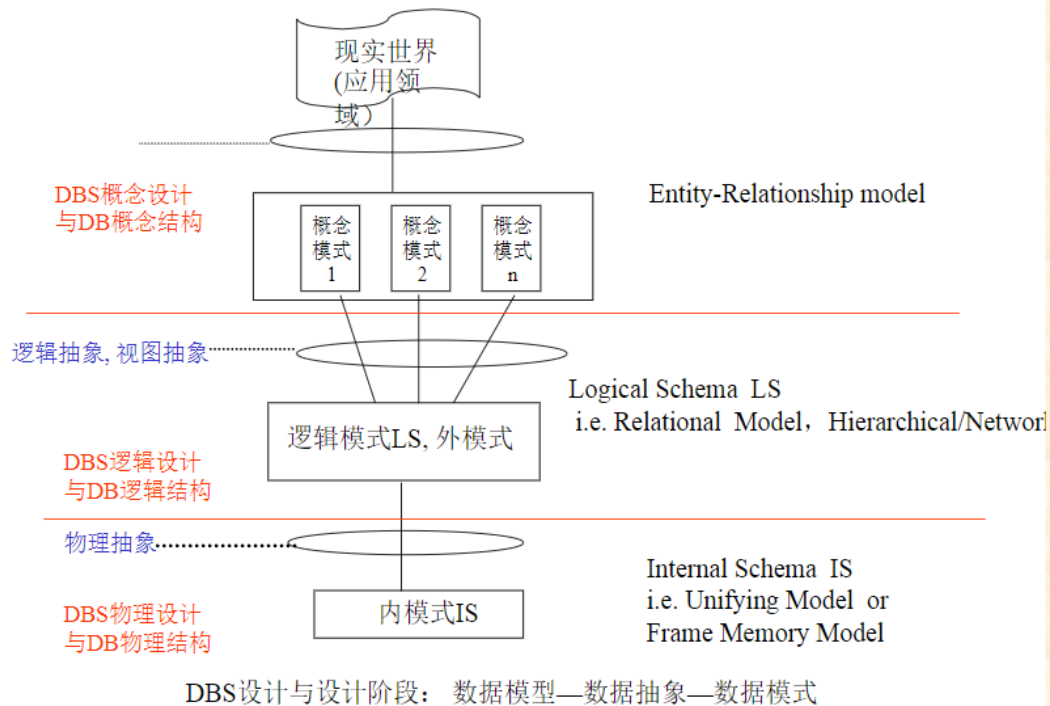
应用程序通常在两个方法访问数据库: ①应用程序编程接口(API,例如ODBC、JDBC、ADO...),这允许发送到数据库的SQL查询②DML调用嵌入宿主语言(宿主语言,如C和Java)



数据库设计

从保持data independence (数据无关性/独立性)角度出发, 根据data models所定义的数据规范形式, 在view、logical、physical三个层次, 采用三种data abstraction方法, 通过DB概念设计、DBS逻辑设计、DBS物理设计三个阶段, 构造面向具体应用领域的DBS的external schema、logical schema、internal schema的集合, 从而得到conceptual DBS、logical DBS、physical DBS的设计结果

■conceptual design ■logical design ■physical design



数据库设计里的规范化Normalization

为了确保数据库里的每个关系都是“好的”，有两种方法：

①建立实体关系模型，用E-R图（第七章）②规范化理论（第八章）

Database Architecture

■Centralized ■Client-server ■Parallel (multi-processor) ■Distributed

DBMS有两个模块

存储管理器storage manager

存储管理器是一个程序模块，它提供数据库中存储的低级数据和应用程序之间的接口，以及提交给系统的查询。存储管理器负责将DML语句转换为低级文件命令，有效地存储、检索和更新数据。存储管理器中的模块有授权和完整性管理器、事务管理器、文件管理器和缓冲区管理器。

查询处理机query processor

The query processor includes ■DDL interpreter（DDL解释器） ■DML compiler（DML编译器） ■Query optimization（查询优化） ■Query evaluation engine（查询执行引擎）

A transaction is a collection of operations that performs a single logical function in a database application。事务是数据库应用程序中执行单个逻辑功能的操作集合

事务属性■原子性，一致性（一致性），隔离性（独立性），持久性（持久性）■DBMS中的事务管理组件（Transaction Management）可确保即使系统发生故障，数据库仍保持一致/正确的状态（例如，并发控制和故障恢复），从而控制并发事务之间的交互，以确保数据库的一致性事务管理

数据库管理员(DBA) DBA的职责包括,但不限于

模式定义、模式和物理组织修改、授予数据访问权限、日常维护(日常维护)

第二章 Introduction to the Relational Model

关系数据库的结构：关系表的集合。关系定义为一系列域上的笛卡尔积的子集。

2.1.1 Basic structure

- Given sets $D_1, D_2, \dots, D_n$, a relation r is a subset of $D_1 \times D_2 \times \dots \times D_n$
 - $r = \{(a_1, a_2, \dots, a_n)\} \subseteq D_1 \times D_2 \times \dots \times D_n$
 - relation $r = \{ \text{tuple} \}$ /*元组
- 定义：关系定义为一系列域上的笛卡尔积的子集

每一个属性都对应一个取值域。属性是无序的。

特殊值null是每一个域的成员。可以表示：①不存在②一个属性值。

所有的域都是原子的（他的每一个元素都是不可分割的），first formal norm.

关系模式

For attributes $A_1, A_2, \dots, A_n$, $R = (A_1, A_2, \dots, A_n)$ is a **relation schema**

- e.g. in Fig. 2.1, **Instructor**-schema = $(ID, name, depart_name, salary)$

- orders of attributes is irrelevant

$r(R)$ is a **relation** on the **relation schema** R

- e.g. *customer* (*customer-name*, *customer-street*, *customer-city*) on *Customer-schema*

Relation instance

- the current values of a relation r at a particular time
- specified by a table

instance 实例

relation关系 relation instance 关系实例

关系表的列(columns)叫属性(attributes), 行(rows)叫元组(tuples)

key 键、码

超码superkey:

形式: 一个或多个属性的集合

作用: 在一个关系中唯一地标识一个元组。

特点: 可能包含无关属性

超码的任意超集也是超码。(通俗讲, 就是同一个关系 r 上, 如果某个属性集合 S 包含超码 K , 即 K 是 S 的子集, 那么 S 也是 r 的超码)

一张表(一个关系)一定有超码, 因为至少所有属性的组合一定是超码, 能唯一确定一个元组

不唯一

候选码candidate key:

形式: 一个或多个属性的集合

作用: 在一个关系中唯一地标识一个元组。

特点: 是超码的一个子集

任意真子集都不可能是超码, 候选码就是最小的超码

没有无关属性

不唯一

主码primary key:

形式: 一个或多个属性的集合

作用: 在一个关系中唯一地标识一个元组。

特点: 是候选码之一

唯一或没有

由数据库设计者指定, 不指定的话表就没有主码

什么是参照完整性约束 **referential integrity constraint**?

参照完整性约束：在参照关系中，任意元组在特定关系上的取值必然等于被参照关系中某个元素在特性关系上的取值。

有向弧：从参照方指向被参照方

关系代数操作（过程化的）

Three types of operations/operators on relations

- *Fundamental*（基本的） operations, *additive*（附加的） operations, and *extended*（可扩展的） operations
- similar to the logical operators in *propositional calculus*（命题演算） and *predicate calculus*（谓词演算）
 - **fundamental** : minimal complete set $\{\neg, \wedge\}$, or $\{\neg, \vee\}$
 - **additive**: implication $\rightarrow$, equivalence $\leftrightarrow$
 - **extended**: $\forall, \exists$

Symbol (Name)	Example of Use
σ (Selection)选择	$\sigma \text{ salary} \geq 85000 \text{ (instructor)}$ Return rows of the input relation that satisfy the predicate.
Π (Projection) 投影	$\Pi ID, salary \text{ (instructor)}$ Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
$\times$ (Cartesian Product) 笛卡尔积	$instructor \times department$ Output all pairs of rows from the two input relations whether or not they have the same value on all attributes that have the same name.
$\cup$ (Union)并	$\Pi name \text{ (instructor)} \cup \Pi name \text{ (student)}$ Output the union of tuples from the <i>two</i> input relations.
$-$ (Set Difference)集合差	$\Pi name \text{ (instructor)} - \Pi name \text{ (student)}$ Output the set difference of tuples from the two input relations.
$\bowtie$ (Natural Join) 自然连接	$instructor \bowtie department$ Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.

注意：

■ Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

■ $\Pi_{A,C}(r)$

集合
需要满足集合特性
不能重复

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2

笛卡尔乘积

■ Relations r ; s :

A	B
α	1
β	2

r

B	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

■ $r \times s$:

A	r.B	s.B	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

自然连接

$R = (A, B, C, D)$

$S = (E, B, D)$

■ Result schema = (A, B, C, D, E)

■ $r \bowtie s$ is defined as:

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$

注：自然连接运算首先形成它的两个参数的笛卡儿积，然后基于两个关系模式中都出现的属性上的相等性进行选择，最后还要去除重复属性。

■ Relations r, s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

■ Natural Join

■ $r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

$$\Pi_{A, r.B, C, r.D, E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$

换名操作

/*将对一个关系instructor中同一个属性salary的不同取值的比较转换为利用换名操作、笛卡尔乘积和选择操作对关系instructor和它的副本 $U_d(\text{instructor})$ 在同一属性salary上取值的比较

Step1.

$\text{instructor} \bowtie \rho_d(\text{instructor})$

Step2. all tuples that have *non-largest salary* values in *instructor* relation:

$$\sigma_{\text{instructor.salary} < d.\text{salary}} (\text{instructor} \bowtie \rho_d(\text{instructor}))$$

第三章 Introduction to SQL

varchar(n). Variable length character strings, with user-specified maximum length n.

smallint. Small integer (a machine-dependent subset of the integer domain type).

numeric(p,d).Fixed point number (定点数), with user-specified precision of p digits (位), with d digits to the right of decimal point (小数点). (ex., numeric(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)

float(n).Floating point number, with user-specified precision of at least n digits

date. Dates, containing a (4 digit) year, month and date
■ E.g. **date** '2001-7-27'
time. Time of day, in hours, minutes and seconds.
■ E.g. **time** '09:00:30' **time** '09:00:30.75'
timestamp: date plus time of day
■ E.g. **timestamp** '2001-7-27 09:00:30.75'
Interval: period of time
■ E.g. Interval '1' day
■ Subtracting a date/time/timestamp value from another gives an interval value
■ Interval values can be added to date/time/timestamp values

Null values are allowed in all the domain types. Declaring an attribute to be not null prohibits null values for that attribute.

create domain construct in SQL-92 creates user-defined domain types 自定义类型

```
create domain person-name char(20) not null
```

```
//建表
create table instructor(
IDchar(5),
name          varchar(20) not null,
dept_name    varchar(20),default() --默认值
salary numeric(8,2),
primary key (ID),
foreign key (dept_name) references department,
check(salary >0) );
/*primary keydeclaration on an attribute automatically ensuresnot
nul*/

//插入
insert into branch
values ('Perryridge', 'Blooklyn', 8000)

//删除
delete from branch
```

drop table命令从数据库中删除关于被删除关系的所有信息(元组和模式)

alter table命令用于添加或删除/删除现有关系

```

drop table r

alter table r add A D
/*where A is the name of the attribute to be added to relation r
and D is the domain of A
all tuples in the relation are assigned nulls the value for the
new attribute*/
e.g. alter table instructor add age char(30)

alter table r drop A
/*一些数据库不支持删除属性*/

```

操作语言

```

select A1, A2, ..., An
from r1, r2, ..., rm
where P
/*如果要消除重复项, 请在select之后插入关键字distinct。*/
/*select *表示选择全部属性*/

select ID, name, salary/12
from instructor
where dept_name = 'Comp. Sci.'

/*
注意事项
from子句中包括多个关系表时, 一定要在where子句中加入连接条件! 防止出现耗时费力的
多表笛卡尔积操作
实际应用中, 对频繁执行的SQL查询, 其from子句中的表的个数不要过多, 如不要超过4
个! 避免耗时费力的多表连接操作。如果频繁执行的SQL查询涉及的查询数据存放在N≥4张
表中, 可以考虑将这N张表中的数据进行合并
*/

select name, course_id
from instructor natural join teaches
where instructor. dept_name= 'Art'

select distinct T.name as instructor_name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp. Sci.'

```

操作符like使用使用两个特殊字符描述的模式

percent (%). The % character matches any substring.

underscore (_). The _ character matches any character.

- Find the names of all instructors whose name includes the substring “dar”.

```
select name
from instructor
where name like '%dar%'
```

- Match the string “100%”

```
like '100 \%' escape '\'
```

in that above we use backslash (\) as the escape (转义) character.

- Pattern matching examples:

- ‘Intro%’ matches any string beginning with “Intro”.
- ‘%Comp%’ matches any string containing “Comp” as a substring.
- ‘_ _ _’ matches any string of exactly three characters.
- ‘_ _ _ %’ matches any string of at least three characters.

- SQL supports a variety of string operations such as

- Concatenation (串接) (using “||”)
- converting from upper to lower case (and vice versa)
- finding string length, extracting substrings, etc.

展示元组的时候，默认按关键字升序展示

```
select distinct name
from instructor
order by name
/*order by name desc*/
/*SQL不主动删除重复，除非用distinct*/
```

sum聚集一般忽略空值，但如果用count(*)计算元组个数，含空值的元组也会被计数

GROUP BY 语句用于结合合计函数，根据一个或多个列对结果集进行分组。

我们拥有下面这个 "Orders" 表:

O_Id	OrderDate	OrderPrice	Customer
1	2008/12/29	1000	Bush
2	2008/11/23	1600	Carter
3	2008/10/05	700	Bush
4	2008/09/28	300	Bush
5	2008/08/06	2000	Adams
6	2008/07/21	100	Carter

现在, 我们希望查找每个客户的总金额 (总订单)。

我们想要使用 GROUP BY 语句对客户进行组合。

我们使用下列 SQL 语句:

```
SELECT Customer,SUM(OrderPrice) FROM Orders
GROUP BY Customer
```

结果集类似这样:

Customer	SUM(OrderPrice)
Bush	2000
Carter	1700
Adams	2000

Attributes in select clause outside of aggregate functions must appear in group by list

```
/* erroneous query */
select dept_name, ID, avg (salary)
from instructor
group by dept_name;
```

-- >ANY,SOME 父查询中的结果集大于子查询中任意一个结果集中的值,则为真

SELECT * FROM T2 WHERE N>ANY(SELECT N FROM T1)

SELECT * FROM T2 WHERE N>SOME(SELECT N FROM T1)

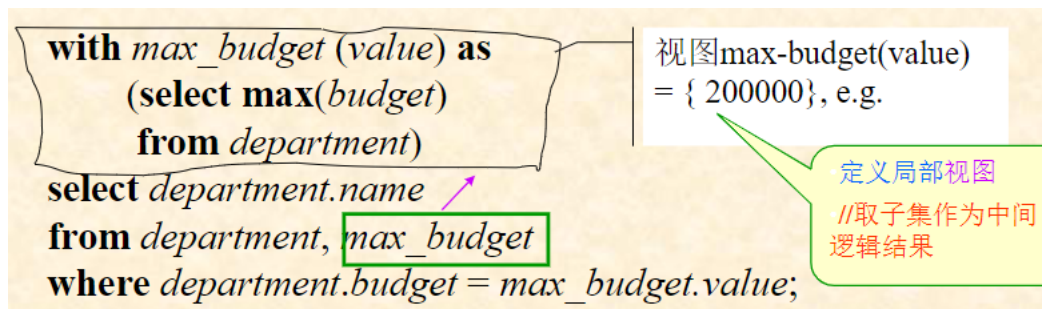
SQL allows < some/all, <= some/all, >= some/all, =some/all, and <>some/all comparisons

SQL uses exists, not exists and except constructs to test whether or not a subquery (, which is a set of tuples) is nonempty or not respectively

- **exists** $r \Leftrightarrow r \neq \emptyset$
- **not exists** $r \Leftrightarrow r = \emptyset$
- **except**: $X - Y$
- $X - Y = \emptyset \Leftrightarrow X \subseteq Y$ (Y包含X)
- **集合包含**, $X \subseteq Y \Leftrightarrow (X - Y) = \emptyset \Leftrightarrow$ **not exists**
 $(X \text{ except } Y)$ $X \subseteq Y \Leftrightarrow$ **not exists** $(X \text{ except } Y)$

unique测试子查询的结果中是否有重复的元组。无就返回true

with子句提供了一种定义临时关系的方法，该临时关系的定义仅对发生with子句的查询可用



```
/*delete all instructors*/
delete from instructor;

delete from instructor
where dept_name= 'Finance';

delete from instructor
where dept name in (
    select dept name
    from department
    where building = 'Watson');

/*插入*/
insert into course
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
或者
insert into course (course_id, title, dept_name, credits)
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

/*更新*/
update instructor
set salary = salary * 1.03
where salary > 100000;
update instructor
set salary = salary * 1.05
where salary <= 100000;
```

不等于: <>

第四章 Intermediate SQL

- date: Dates, containing a (4 digit) year, month and date
 - e.g. date '2017-7-27'
- time: Time of day, in hours, minutes and seconds
 - e.g. time '09:00:30.75'
- 时间戳 ■ timestamp: date plus time of day
 - e.g. timestamp '2017-7-27 09:00:30.75'
- interval: period of time
 - if x and y are of type date, then $x-y$ is an interval whose value is the number of days from date x to date y
 - e.g. interval '1' day='2017-7-27' - '2017-7-28'

- cast e as t
 - convert a character string (or string valued expression) e to the type t , e.g. date/time/timestamp
 - e.g. cast '2017-07-20' as date
 - e.g. cast '09:00:30.75' as time
- For a *day* or *time* value d , its individual fields can be extracted
 - e.g. extract (year from 2017-07-20) =2017
 - e.g. extract (timezone_hour from 2017-7-27 09:00:30.75) =09

blob: binary large object (二进制大型对象)

clob: character large object

XML: 存半结构化/非结构化数据

cast, i.e. convert, values of one type to another domain

cast(department.budgtto numeric(12.2))

```
-- CAST Syntax:
CAST ( expression AS data_type [ ( length ) ] )

-- CONVERT Syntax:
CONVERT ( data_type [ ( length ) ] , expression [ , style ] )
```

create domain degree_level varchar(10)

constraint degree_level_test(可缺省)

check (value in ('Bachelors', 'Masters', 'Doctorate'));

select* from course join prereq on course.course_id=prereq. course_id(连接条件)

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

■ *course*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

□ *prereq*

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

外部连接

左部连接

□ *course* **natural left outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

右部连接

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

完全连接

course **natural full outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

■ **select ***

from *course* **natural full outer join** *prereq*

内部连接和外部连接的区别：

内连接：只显示两表id匹配的

左外连接：显示join左边的表的所有数据（不管两表是否匹配），对于不匹配的部分都用NULL显示

右外连接：与左外连接相反，显示join右边的表的所有数据

■ *course* **inner join** *prereq* **on**

course.course_id = *prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

□ What is the difference between the **inner join** mentioned above, and a **natural join**?

□ *course* **left outer join** *prereq* **on**

course.course_id = *prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	<i>null</i>	<i>null</i>

Views（视图） virtual relation/table

Materialized views（实体化/物化视图）

```
create view faculty as
select ID, name, dept_name
from instructor

--然后还可以基于视图查询
select name
from faculty
where dept_name = 'Biology'
```

Most SQL实现只允许更新简单的views

from子句只有一个数据库关系。

select子句只包含关系的属性名，没有任何表达式、聚合或不同的规范。

没有在select子句中列出的Any属性可以设置为null

查询没有group by或having子句。

在 SQL 中增加 HAVING 子句原因是，WHERE 关键字无法与合计函数一起使用。

SQL HAVING 实例

我们拥有下面这个 "Orders" 表:

O_Id	OrderDate	OrderPrice	Customer
1	2008/12/29	1000	Bush
2	2008/11/23	1600	Carter
3	2008/10/05	700	Bush
4	2008/09/28	300	Bush
5	2008/08/06	2000	Adams
6	2008/07/21	100	Carter

现在，我们希望查找订单总金额少于 2000 的客户。

我们使用如下 SQL 语句:

```
SELECT Customer,SUM(OrderPrice) FROM Orders
GROUP BY Customer
HAVING SUM(OrderPrice)<2000
```

■ **select** *instructor.**
from *instructor , teaches*
where *instructor.ID = teaches.ID*

Integrity Constraints:完整性约束

Referential Integrity关联/参照完整性：一个好的db设计应该确保任何关系模式R2(及其任何元组)只能通过外键引用其他关系模式r1

和主码不同，候选码允许为空。默认情况下，外码引用被引用表的主码属性1.

```
■ create table course (  
    course_id char(5) primary key,  
    title      varchar(20),  
    dept_name varchar(20) references department  
)  
■ create table course (  
    ...  
    dept_name varchar(20),  
    foreign key (dept_name) references department  
        on delete cascade  
        on update cascade,  
    ...  
)  
■ alternative actions to cascade: set null, set default
```

被引表更新
会影响引表
级联：- 自违背完整性约束
级联删除
级联更新

关系模式建立时，除了多对多，都用一个含另外一个的主键（外键引用），多对多才建第三者（新表）。外键不必下划线。一对多不是全参与，也要建新表。全部参与尽量合并，不建表。

An assertion is a predicate expressing a condition that we wish the database always to satisfy

断言是一种谓词，表示我们希望数据库始终满足的条件。但SQL Server不满足。

create assertion check

```
■ create assertion credits_earned_constraints check  
    ( not exists (select ID  
        from student  
        where tot_cred  
            <> select sum(credits)  
                from takes natural join course  
                where student.ID=takes.ID  
                and grade is not null and grade <> 'F')  
    )
```

grant select on instructor to U1, U2, U3

- create role instructor;
- **grant** *instructor* **to** Amit;
- Privileges can be granted to roles:
 - **grant select on** *takes* **to** *instructor*;
- Roles can be granted to users, as well as to other roles
 - **create role** *teaching_assistant*
 - **grant** *teaching_assistant* **to** *instructor*;
 - *Instructor* inherits all privileges of *teaching_assistant*
- Chain of roles
 - **create role** *dean*;
 - **grant** *instructor* **to** *dean*;
 - **grant** *dean* **to** Satoshi;

Authorization on Views

- **create view** *geo_instructor* **as**
(**select** *
from *instructor*
where *dept_name* = 'Geology');
- **grant select on** *geo_instructor* **to** *geo_staff*
- Suppose that a *geo_staff* member issues
 - **select** *
from *geo_instructor*;
- What if
 - *geo_staff* does not have permissions on *instructor*?
 - creator of view did not have some permissions on *instructor*?



Other Authorization Features

- **references** privilege to create foreign key
 - **grant reference** (*dept_name*) **on** *department* **to** Mariano;
 - why is this required?
- **transfer of privileges**
 - **grant select on** *department* **to** Amit **with grant option**;
 - **revoke select on** *department* **from** Amit, Satoshi **cascade**;
 - **revoke select on** *department* **from** Amit, Satoshi **restrict**;
- Etc. read Section 4.6 for more details we have omitted here.

第五章 Advanced SQL

JDBC (Java Database Connectivity) works with Java

Open a connection, named as *conn*

create a SQL "statement" object on the connection *conn*, its statement handle is *stmt*

retrieve (检索) into a ResultSet object *rset*

ODBC (Open Database Connectivity) works with C, C++, C#, and Visual Basic

Other API's such as ADO.NET sit on top of ODBC

Embedded SQL

交互式SQL只能进行DB的访问操作，不能对DB访问结果进行进一步的数据处理

Embedded SQL将SQL的数据库访问功能与C语言等宿主语言的数据处理能力相结合，提高了数据应用系统的能力

识别到预处理器的嵌入式SQL请求: EXEC SQL ;

在执行任何SQL语句之前，程序必须先连接到数据库。EXEC-SQL connect to server user user-name using password;

可以在嵌入式SQL语句中使用宿主语言的变量。它们前面有一个冒号(:)以区别于SQL变量:
:credit_amount

游标cursor: declare ccursor for

利用Embedded SQL进行查询时，查询结果有可能包括多个元组，此时无法直接将多个元组通过共享变量赋值传递给宿主程序

系统开辟专门working区域存放SQL查询的结果关系（也称作中间关系），并利用查询游标c指向此区域。宿主程序根据c指向的查询结果关系集合，使用open, fetch, close依次获取结果关系中的各元组

```
EXEC SQL BEGIN DECLARE SECTION
    int credit_amount;
    char si, sn;
EXEC SQL END DECLARE SECTION
    credit_amount := input-from-screen-by-users
EXEC SQL
    declare c cursor for
        select ID, name
        from student
        where tot_cred > :credit_amount
END-EXEC
EXEC SQL open c END-EXEC
EXEC SQL fetch c into :si, :sn END-EXEC
EXEC SQL close c END-EXEC
```

可以用循环语句依次取走全部查询结果

shared variable

SQL Procedures

```
create procedure dept_count_proc (in dept_name varchar(20), out
d_count integer)
begin
select count(*) into d_count
from instructor
where instructor.dept_name = dept_count_proc.dept_name
end
```

函数、存储过程预先生成对应的查询执行计划（类似于目标代码），存储在DBMS中，应用程序直接调用，不需再进行查询处理和优化（i.e.编译）

Trigger（触发器）作为数据库修改的副作用，DBMS自动执行的一条语句

Trigger is an event-condition-action model based integrity definition, checking, remedy（更正） mechanism

第六章 Fromal Relational Query Language

关系代数

fundamental operations

- Six basic operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - rename: ρ
- The operators take one or two relations as inputs and produce a new relation as a result.

选择、投影、并集、差集、笛卡尔积、重命名

select *

from r

where p

p:selection predicate 选择谓词

- renaming r or E as X

additional operations

- Additional operations
 - $\cap$, set-intersection (集合交)
 - ∞ , $\bowtie$ natural-join (自然、外连接)
 - $\leftarrow$, assignment (赋值)

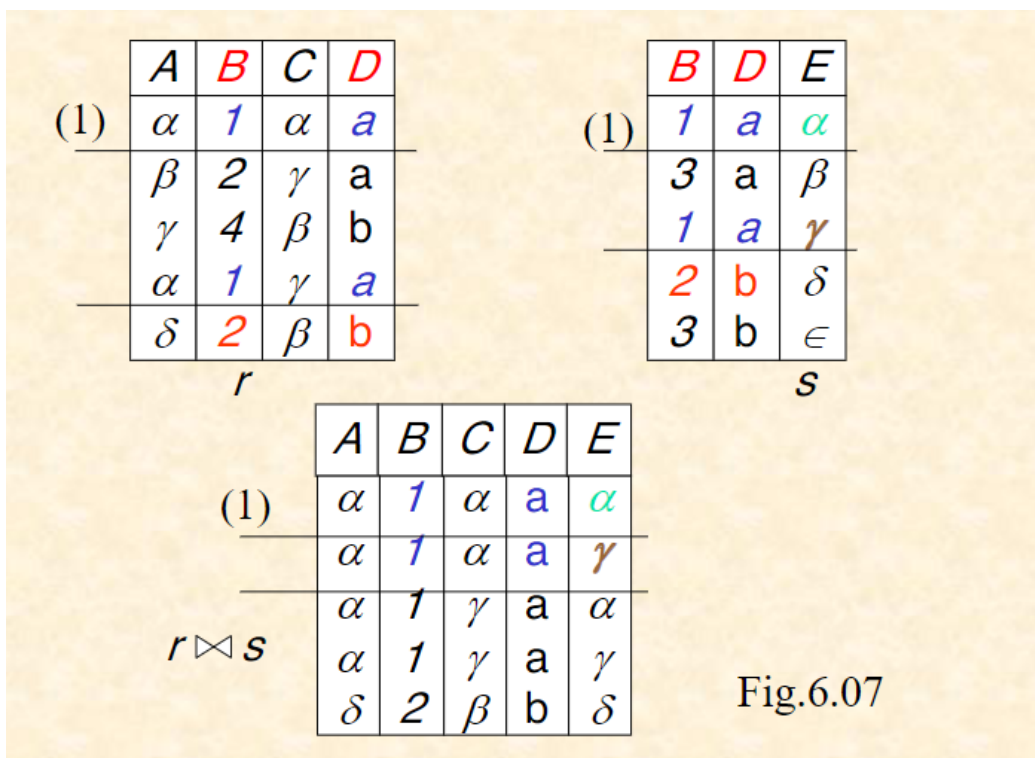


Fig.6.07

- $r \bowtie (s \bowtie t) = (r \bowtie s) \bowtie t$
- if $R \cap S = \Phi$, then $r(R) \bowtie s(S) = r(R) \times s(S)$

将 $(\bowtie)$ 关系代数表达式 E 分配给一个关系变量 var_r , 即将 E 表示为 var_r . $\text{var}_r \leftarrow E$

内连接: 保留有匹配、有相关性的

外连接: 没有匹配成功的也保留下来, 左外连接保留左边, 右外连接保留右边

双向外连接

ID	course_id	sec_id	semester	year	name	dept_name	salary
10101	CS-101	1	Fall	2009	Srinivasan	Comp. Sci.	65000
10101	CS-315	1	Spring	2010	Srinivasan	Comp. Sci.	65000
10101	CS-347	1	Fall	2009	Srinivasan	Comp. Sci.	65000
12121	FIN-201	1	Spring	2010	Wu	Finance	90000
15151	MU-199	1	Spring	2010	Mozart	Music	40000
22222	PHY-101	1	Fall	2009	Einstein	Physics	95000
32343	HIS-351	1	Spring	2010	El Said	History	60000
33456	null	null	null	null	Gold	Physics	87000
45565	CS-101	1	Spring	2010	Katz	Comp. Sci.	75000
45565	CS-319	1	Spring	2010	Katz	Comp. Sci.	75000
58583	null	null	null	null	Califieri	History	62000
76543	null	null	null	null	Singh	Finance	80000
76766	BIO-101	1	Summer	2009	Crick	Biology	72000
76766	BIO-301	1	Summer	2010	Crick	Biology	72000
83821	CS-190	1	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-190	2	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-319	2	Spring	2010	Brandt	Comp. Sci.	92000
98345	EE-181	1	Spring	2009	Kim	Elec. Eng.	80000

Figure 6.18 Result of $\text{teaches} \bowtie \text{instructor}$.

注: 左右都有null, 双向外连接
有外连接, 保留不也

extended operations

Generalized Project (一般化/扩展投影操作)

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

```
Select   {  $A_1, A_2, \dots, A_i$ , }  $ag\_fun(A_{i+1}), \dots, ag\_fun(A_{i+k})$   
From     $r_1, r_2, \dots, r_m$   
Where    $P1$   
{ Group by  $A_1, A_2, \dots, A_i$   
  { having  $P2$  } }
```

notes:

- $P1$ is defined on all attributes of $r_1, r_2, \dots, r_m$
- $P2$ is defined on $A_1, A_2, \dots, A_i, \dots, A_j, \dots, A_k, \dots, A_n$, as the constraints on the group formed by **Group by** A_1, A_2, A_j

元组关系演算

域关系演算

select *
from course join prereq on course.course_id=prereq.course_id

这个join和inner join应该一样？都会保留相同的两列

[自动回复]你自己掂量掂量学习和你谁重要

17:16:37

yep

okk

他和自然连接

一样吗

不一样

course_id	title	dept_name	credits	prereq_id	course_id
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

自然连接没有最后一列

第七章 Database Design and E-R Model

aggregation 聚集

关系模式建立时，除了多对多，都用一个含另外一个的主键（外键引用），多对多才建第三者（新表）。外键不必下划线。一对多不是全参与，也要建新表。全部参与尽量合并，不建表。

designing of DB schemas , in accordance with the three-level of data abstract (refer to Fig. 1.1)

conceptual design

logical design, at the logical level and view level 逻辑阶段有二维表输出

physical design, at the physical level

数据库应用系统DBAS设计：数据组织与存储设计、数据访问与处理设计、应用设计三条设计主线，分别用于设计数据库、数据库事务和应用程序。分为概念设计、逻辑设计、物理设计三个步骤。

并发用户数=总用户数*10%

E-R图

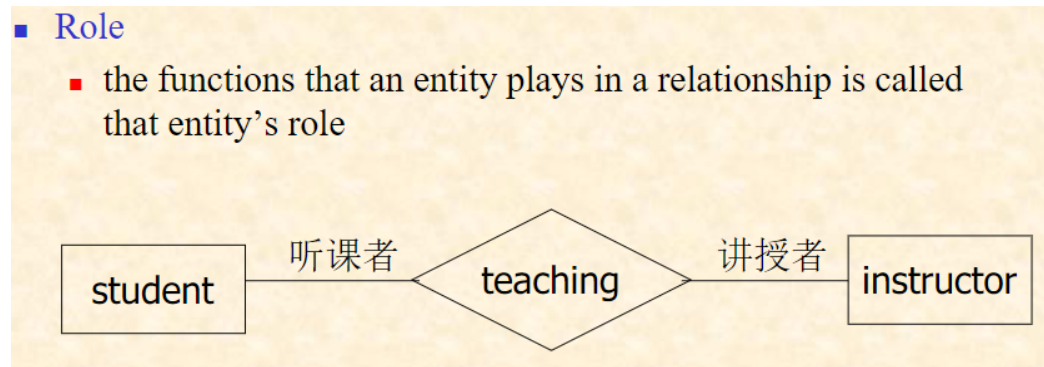
实体用属性的集合表示，一个实体包含了他所有属性的所有值。

关系指几个实体的关联关系。关系集是同种类型关系的集合。

对于one-many的关系而言，one这一段确定了，我们并不能确定这段关系。但many一端确定了，我们就可以知道确切的关系。因此，我们一般用many一端的主属性，来避免冗余属性。

大多数的关系都是二元的，哪怕遇到三元的，我们也会用多个二元去表示三元。

角色



属性也有很多种类：简单属性和复合属性，单值属性和多值属性，推导属性。

Null value for a attribute means

①the attribute “not applicable” for the entity, not existing ②the value for the attribute exists, but is “unknown”

Integrity constraints in E-R model, on E or R

①键

键是属性集，the values of these attributes in one entity can be used to uniquely distinguish this entity from others, or the values of these attributes in one relationship are used to uniquely identify the relationship.

superkey（超码）、candidate key（候选码）、primary key（主码）

候选码是最小、不冗余的超码

■ The **super**_key for R

- $\text{primary_key}(E_1) \cup \text{primary_key}(E_2) \dots \cup \text{primary_key}(E_n)$

构造关系表时，如果有多个候选键，最好选取数值型（int, float）候选键作为关系表主键，便于提高基于主键的查询速度—不要选字符串型属性，如varchar、datetime

对于one-many的关系而言，one这一段确定了，我们并不能确定这段关系。但many一端确定了，我们就可以知道确切的关系。因此，我们一般用many一端的主属性，来避免冗余属性。



■ R is *many-to-one from A to B*, !!

$\text{primary_key}(R) = \text{primary_key}(A)$

■ R is *one-to-many*, !! ▶

$\text{primary_key}(R) = \text{primary_key}(B)$

■ R is *one-to-one*, ▶

$\text{primary_key}(R) = \text{primary_key}(A)$

or: $\text{primary_key}(R) = \text{primary_key}(B)$

②映射基数mapping cardinalities

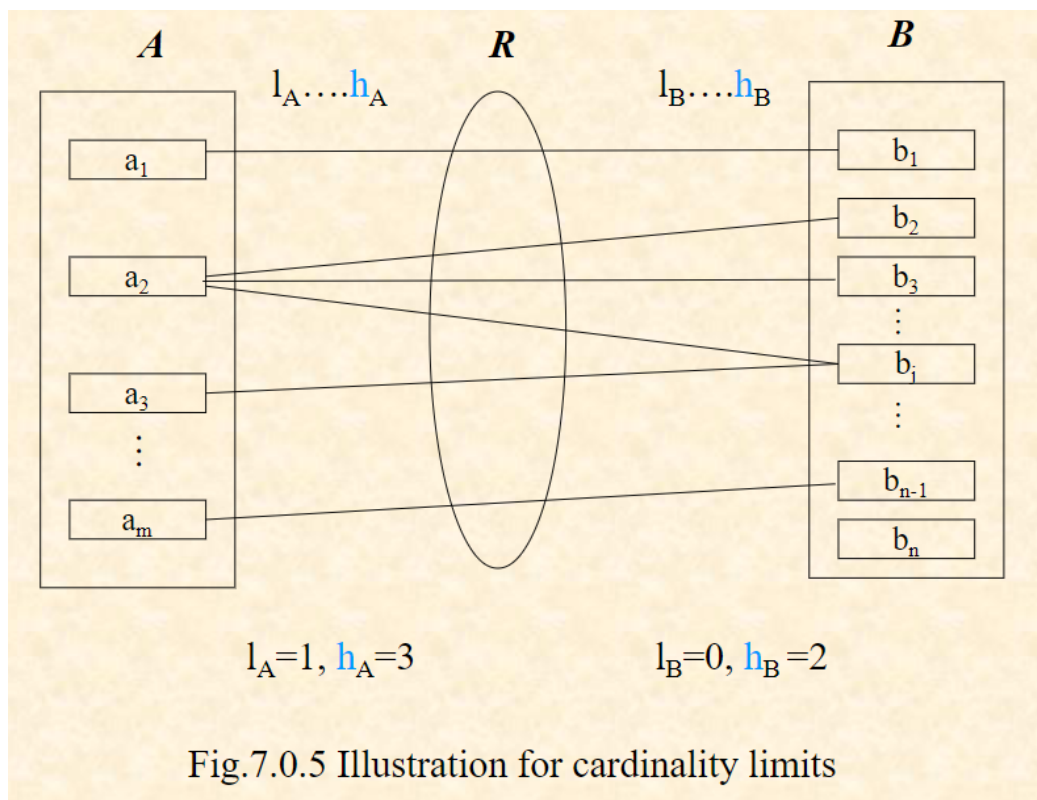
one to one, one to many, many to one, many to many(箭头指向one，无箭头指向many)
——at most

③参与约束participation-constraints——定性

部分参与约束partial participation、全部参与约束total participation

④基数限制cardinality limits——定量

A参与R的技术下界lA和上界hA：对A中的每个实体，B中至少有lA个实体通过R与A相关联，B中至多有hA个实体通过R与A相关联



按最大 h_A 判断是one-one还是一many, $\langle h_B, h_A \rangle$ 表示了联系R的从A到B的映射基约束。

最小 l_A 如果等于0, 那么A就是部分参与; 最小 l_A 如果大于0, 那么A就是全部参与

完整性约束: <https://www.cnblogs.com/thescentedpath/p/constraint.html>

弱实体集: 没有主码的实体集

弱实体集 E_1 只能通过另一个(强)实体集 E_2 来区分, 该实体集 E_2 称为标识(所有者)实体集, 该实体集与 E_1 具有关联/关系R

- E_2 owns E_1 , E_1 must be related to E_2 via an instance of R, called the *identifying relationship* (标识性联系).
- R is *many-to-one* (or *one-to-one*) from the weak entity set E_1 to the identity set E_2

一般弱实体集都是全部参与到和强实体集的关系中去。弱实体E-R图中双框, 他参与的关系也是双框。全部参与是双线, 部分参与是单线。

弱实体集有自己的discriminator(分辨符, 也叫partial key), 下画虚线。弱实体集利用强实体集的主码和他自身描述符来标识。

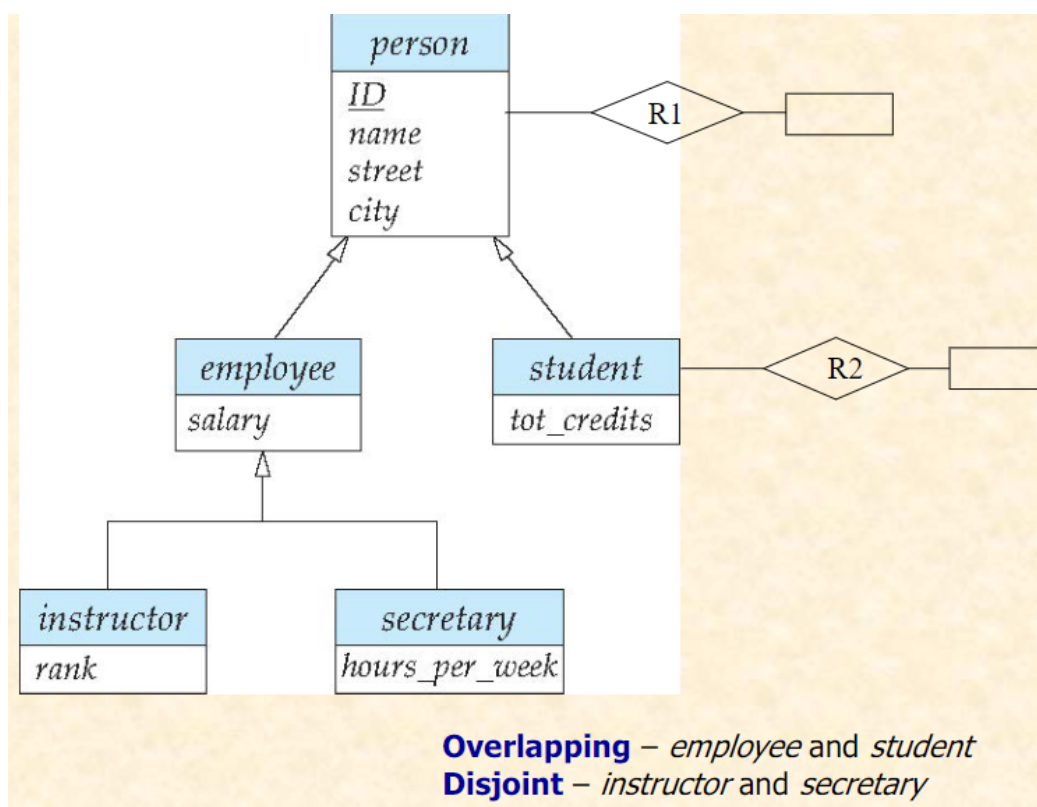
- **Rectangles (矩形)** represent entity sets
- **Diamonds (菱形)** represent relationship sets
- **Lines** link attributes to entity sets and entity sets to relationship sets.
- **Ellipses (椭圆)** represent attributes
 - **double ellipses** represent **multivalued attributes**
 - **Dashed (虚线) ellipses** denote **derived attributes (派生属性)**
- **Underline** indicates primary key attributes
- Double rectangles represent weak entity sets

Object-oriented(OO) E-R

specialization (特化, 特殊化, 例化) —— 三角形标记ISA

在一个实体集合中指定子组的自顶向下设计过程, 根据实体集合中实体之间的差异, 每个子组都与其他实体有区别

overlapping: 部分重叠 disjoint: 不相交



generalization (概括化, 泛化, 普遍化)

一种自下而上的设计过程, 它将许多具有相同特性的实体集组合成一个更高层次的实体集

for the entity sets A and B, if A is the generalization of B, i.e. B is the specialization of A, then A is the superclass of B, and B is the subclass of A

attributes inheritance (属性继承)

较低级别的实体集继承它所链接到的较高级别的实体集的所有属性和关系参与

Constraints on Specialization/Generalization

Completeness constraint

total: an entity must belong to one of the lower-level entity sets, e.g. student generalization

partial: an entity need not belong to one of the lower-level entity sets, e.g. .person specialization

disjoint: a high-level entity can belong to only one lower-level entity set, i.e., $L\text{-entity-set-1} \cap L\text{-entity-set-2} = \Phi$

overlapping: a high-level entity can belong to more than one lower-level entity sets

■ depicted in E-R diagram as

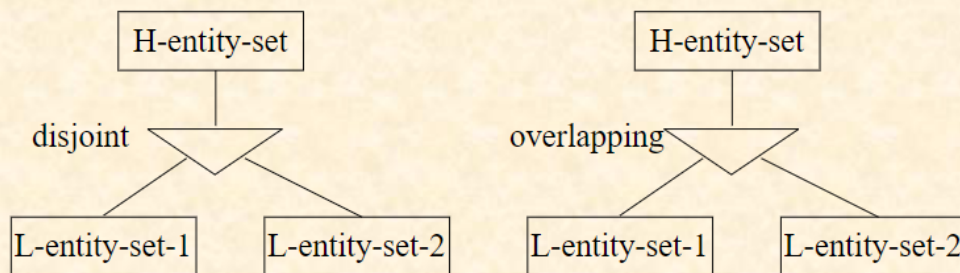


Fig.7.0.11 disjoint and overlapping in generalization

Aggregation: 把联系建模成一个聚集（实体集）to express relationship among relationships, 联系上的联系

关系模式Relational Schemas

Table=relation

逻辑设计的两步：①初始模式集：由E-R图直接转换而来的（需要规范化）②关系模式集的规范化

强实体集直接对应这个table，完美一致。

弱实体集本省必须关联一个强实体集存在，因此标识的时候也必须把它关联的强实体集的主码标识也要转化成表的一个属性

属性集和实体集不同。实体集直接对应一个table就行。

简单属性集可以看做属性集。对于关系数据库来说就需要有一个基本要求：每一个属性是原子的。因此对于复合属性集，不能直接转成一个table，因为复合属性可以再分。

直接应用复合属性的每一个子属性，每一个子属性成一行（分别成行）。

而对于多值属性（数据库不支持数组类型），每一个值要单独成行。此外，将多值属性单独创一张表（和其他属性分开），然后通过他们的主码标识和其他属性联系起来。

联系集的转化：根据不同的联系基数和参与约束来实现不同的转化方式。

多张表的联系方式：①关联关系隐含在属性集里面——外码标识②直接联系

对于联系集，如果他自身也有一些标识性属性，也可以把它加入到联系集的表里面去。

把one这一端的主码发给many，作为其外码进行联系。（one-one, one-many, many-one)

此外，对于部分参与的many约束，我们最好单独创建一张表——避免空值问题的产生。为了进一步避免空值问题，我们最好不要留空值，设置一个默认值。

Reduction of Specialization/Generalization

方法①：每个子类都单独建立一张表，通过父类主码标识——获取信息需要访问两个关系，一个对应于低级模式，另一个对应于高级模式。

方法②：把共性特征也作为子类属性构建出来——可能会有信息冗余。

特化约束、概化约束，来考虑合适的方式。

特化、概化约束种类：

相交不相交约束

高层实体集至多属于一个子类——不相交、不重叠

高层实体集既可能属于A子类，也可能属于B子类，或者子类间没有明显差距

全部部分约束

全部特化、概化

部分特化、概化——有一部分不属于任何一个子类

对于三个实体集，建三表

overlapping OR partial

父类子类分别建表，三张表表达

共性标识放父类，子类里面放父类主码标识和子类自身的其他

如果概化是不相交且完全的

disjoint AND complete

由于高层属性都能属于一个子类，没有必要单独建表，信息用子类标识就可以啦。